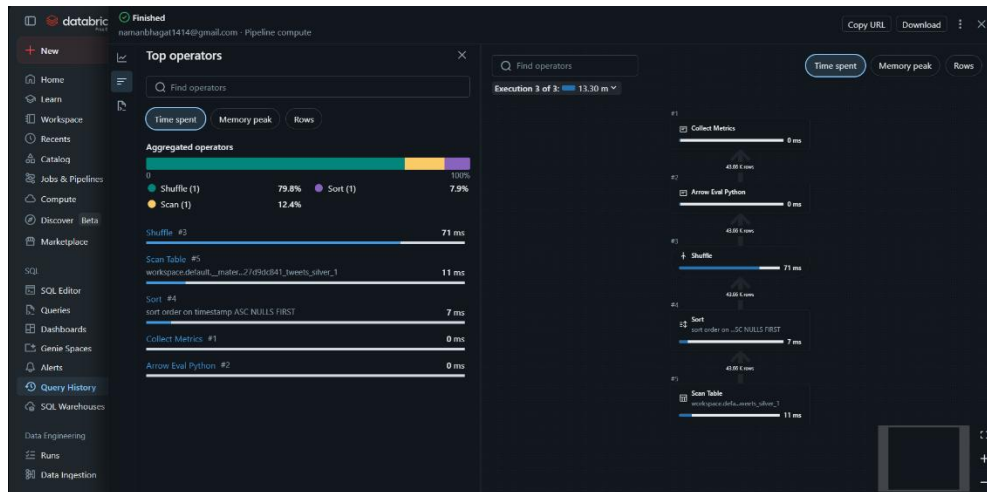


Performance Analysis

1. Shuffle

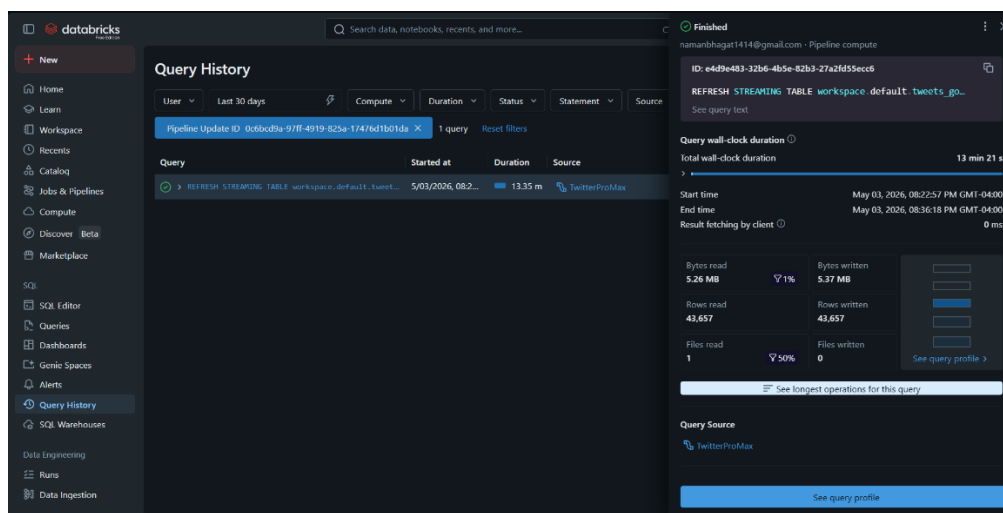


The most noticeable performance cost in the Gold transformation comes from shuffle operations. Shuffle happens when Spark redistributes data across executors, which involves network communication and coordination overhead.

From the execution view, shuffle dominates the runtime, taking up **the majority of the execution time (~75–80%)**. Even though the dataset is relatively small (~43K rows), the pipeline still spends most of its time moving data rather than processing it.

Shuffle occurs because Spark must reorganize data across partitions, which involves network transfer, task coordination, and partition-level execution. These operations are significantly more expensive than computation (such as model inference). In this pipeline, shuffle is not driven by complex transformations but still dominates execution, indicating the workload is primarily limited by data movement rather than compute. Performance could be improved by tuning partition count, avoiding unnecessary repartitioning, and minimizing the amount of data shuffled before inference. Overall, reducing shuffle overhead would have the greatest impact on runtime.

2. Serialization



A key aspect of the Gold layer performance is the overhead introduced by running model inference using a Python-based MLflow UDF. Instead of staying entirely within Spark's JVM execution, each record is sent to the Python environment for prediction and then returned back.

Since the Gold table continuously reads from tweets_silver, this process happens for every incoming tweet. This creates a serialization/deserialization step between Spark and Python.

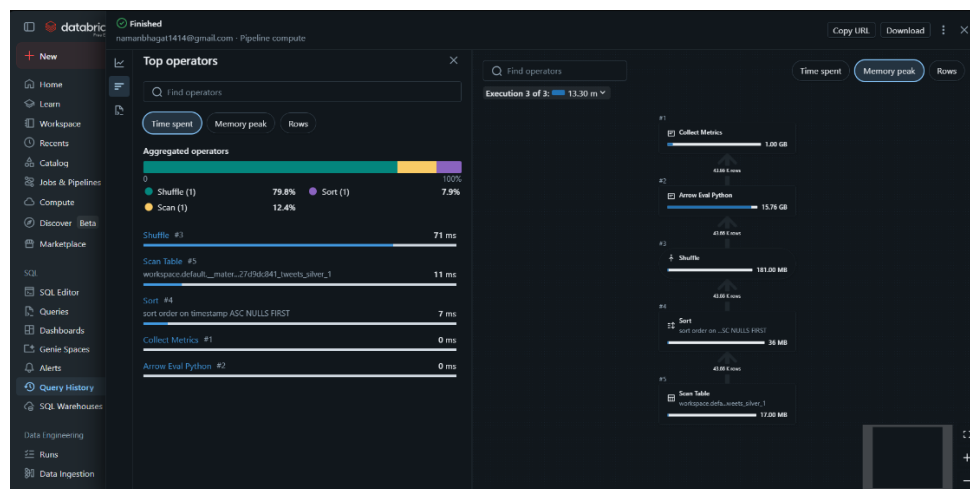
To reduce this cost, the pipeline uses Apache Arrow, which batches records before sending them to Python. This is visible in the execution plan through the **Arrow Eval Python** operator. Because of batching, the system avoids processing rows individually, which significantly reduces overhead.

Looking at the runtime metrics, the overall query processed around **43K rows** with roughly **5 MB of data**, finishing in about **13 minutes**. Despite the use of Python UDFs, the execution breakdown shows almost no time spent in the Python operator itself.

This indicates that the model inference step is relatively efficient and not slowing down the pipeline in a meaningful way. Even though serialization exists, it does not significantly affect total runtime.

One possible improvement could be switching to a fully Spark-native inference method or increasing Arrow batch sizes. However, based on current observations, optimizing serialization would not lead to major gains compared to other parts of the pipeline.

3. Spill



To better understand memory behavior, different partitioning strategies were considered to observe how they impact execution. Increasing parallelism generally improves throughput for distributed workloads, but it also increases memory usage due to more active tasks. From the Spark UI "Memory peak" view, the Arrow Eval Python operator reached approximately **15.7 GB**, which was significantly higher than other operators such as Shuffle (~180 MB) and Scan (~17 MB). This indicates that model inference, combined with Arrow batching, is the most memory-intensive part of the pipeline.

Despite this, no spill to disk occurred, suggesting that the chosen level of parallelism was still within the available memory limits. This implies that the current configuration achieves a balance between parallel execution and memory usage. Based on this observation, further increasing partitions could risk higher memory pressure, while reducing partitions may help lower shuffle overhead without introducing spill. Therefore, tuning partition count is an important trade-off between performance and memory efficiency.